



Universidad
Indoamérica

UNITAT 7 · FONAMENTS DE SISTEMES OPERATIUS

Fils i concurrència

Universidad Tecnológica Indoamérica

Laboratori TD6

Un fil: una execució seqüencial dins d'un procés

- També «procés lleuger»: diverses execucions en un procés
- Tots comparteixen el **mateix espai d'adreces**
- **No hi ha protecció de memòria** entre fils
- Un procés = memòria virtual (protecció) + fils (execució paral·lela)

Què es comparteix, amb adreces reals

procés · 8 thread(s)

codi

dades: shared

0x650ff476602c = 8

heap

descriptors 0 1 2

compartit per tots els fils

fil 1

pila

local 0x31fee74

registres: PC, SP...

fil 2

pila

local 0x19fbe74

registres: PC, SP...

fil 3

pila

local 0xbffee74

registres: PC, SP...

fil 4

pila

local 0x29fde74

registres: PC, SP...

fil 5

pila

local 0x21fce74

registres: PC, SP...

fil 6

pila

local 0x11fae74

registres: PC, SP...

fil 7

pila

local 0x09f9e74

registres: PC, SP...

fil 8

pila

local 0xb7fde74

registres: PC, SP...

piles separades 8,004 MiB (8 MiB + 4 KiB de guarda)

Programes multifil

- Sistemes encastrats: un programa, operacions concurrents
- El mateix nucli: milers de fils del nucli
- Bases de dades i servidors de xarxa (unitat 8)
- Programes paral·lels: OpenMP, un fil per nucli

Avui també

Navegadors (fils per pestanya i per tasca), motors de jocs, inferència de models de llenguatge a la CPU (unitat 9) i Python, que des de la versió 3.13 ofereix una variant sense el pany global (GIL).

Crear un fil és més barat... quant?

Sortida real

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

- Cap pàgina per copiar: només una pila nova i una estructura de tasca
- El sistema planifica cada fil: **no suposis cap ordre d'execució**

Crear, acabar i esperar un fil

C

```
int pthread_create(pthread_t *t, const pthread_attr_t *attr, /* NULL: defaults */
                  void *(*start)(void *), void *arg);
void pthread_exit(void *retval); /* = return retval from start; never returns */
int pthread_join(pthread_t t, void **retval); /* PTHREAD_CANCELED if t was cancelled */
int pthread_detach(pthread_t t); /* nobody will join t: freed when it ends */
```

- `attr` tria, per exemple, *joinable* (per defecte) o *detached*: es configura amb `pthread_attr_init` i `pthread_attr_setdetachstate`
- Un fil que acaba **no allibera** la seva pila ni el seu estat fins que algú l'espera (o si està separat): si no, fuga de memòria, com un zombi
- `join` suspèn qui el crida fins que `t` acaba i en recull el `retval`; un fil separat no es pot esperar
- A Linux, `pthread_create` fa servir la crida `clone`: el mateix espai d'adreces, els mateixos descriptors

pthread_create, pthread_exit, pthread_join

C tds/td6/threads.c

```
for (int i = 0; i < NTHREADS; i++) {
    int rc = pthread_create(&tids[i], NULL, start, (void *)(&i));
    exit_if(rc != 0, "pthread_create"); /* pthread functions return the error code */
}
for (int i = 0; i < NTHREADS; i++)
    exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
```

Sortida real

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
...
```

Correccions (diapositives 137–140): `-pthread`, no `-lthread`; `pthread_*` retorna 0 o el número d'error, no `-1` amb `errno`.

Fils i senyals

Els senyals pertanyen a **tot el procés**. Per als fils:

PROCÉS	FILS
<code>kill()</code>	<code>pthread_kill()</code>
<code>sigprocmask()</code>	<code>pthread_sigmask()</code>
<code>sigsuspend()</code>	<code>sigwait()</code> (un fil dedicat a esperar senyals)

Concurrencia i memòria compartida: mals de cap

La major part del temps els fils toquen dades diferents i l'ordre no importa. Però amb `x = 1` inicialment:

```
c
/* thread 1 */      /* thread 2 */
x = x + 2;           x = x * 3;
```

Quins valors pot prendre `x`? → **no determinista**.

- Val igual per a fils i per a processos amb memòria compartida (unitat 6)
- Errors difícils de reproduir: apareixen una vegada cada milions d'execucions
- Un exemple pitjor (diapositiva 146):
`while (x < 1000) x++;` contra
`while (x > -1000) x--;` pot guanyar qualsevol dels dos... o pot no acabar mai

Els 20 intercalats, un per un

fil 1: $x = x + 2$;

1. $r1 = x$

2. $r1 = r1 + 2$

6. $x = r1$

fil 2: $x = x * 3$;

3. $r2 = x$

4. $r2 = r2 * 3$

5. $x = r2$

memòria

$x=1$

$x=1$

$x=1$

$x=1$

$x=3$

$x=3$

resultat dels 20 intercalats

$x = 3: 18$

$x = 9: 1$

$x = 5: 1$

real, TD6 (8 fils, 100.000 entrades):

tickets_race: 103.195 venudes (tickets = -7)

tickets_mutex: 100.000 venudes (tickets = 0)

tickets_sem: 100.000 venudes (tickets = 0)

Operacions atòmiques

- S'executen del tot o gens: **indivisibles**
- Típicament: un accés a memòria de mida nativa, una instrucció simple
- No atòmic: `x++` (llegir-sumar-escriure), accedir a 64 bits en una màquina de 32 bits

En C modern

Des de C11, que dos fils accedeixin a la mateixa variable sense sincronització (amb almenys una escriptura) és una **corsa de dades: comportament indefinit**. Per a operacions atòmiques: `_Atomic`, `<stdatomic.h>` o `__atomic_add_fetch` (el fa servir `shm_fork`, unitat 6). `volatile` no fa res atòmic.

Massa llet: exclusió mútua per programari

FIL 1	FIL 2
mira a la nevera: no hi ha llet	
va a la botiga	mira a la nevera: no hi ha llet
compra llet	va a la botiga
torna	compra llet... massa!

- **Sincronització:** fer servir operacions atòmiques per coordinar fils
- **Exclusió mútua:** només un fil fa l'acció cada vegada
- **Secció crítica:** el codi que només pot executar un fil cada vegada
- Solucions 1–3: tancar la nevera amb clau, una nota, una nota per fil... i els seus errors

Peterson i Dekker: correctes... sobre el paper

C

```

/* Peterson, thread 1 */
postit_1 = true; turn = 2;
while (postit_2 && turn == 2)
    ;                               /* busy waiting */
if (!milk) milk++;                 /* critical section */
postit_1 = false;

```

- Correctes si els accessos a memòria són atòmics i **ordenats**
- Però el compilador i el processador **reordenen** (diapositiva 154)
- I fan espera activa: cremen CPU

Sortida real

```

$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
...
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
...

```

Cada solució, cada estat

fil 0

```

postit_0 = 1
turn = 1
/* sense barrera */
loop: q = postit_1
if (!q) goto cs
t = turn
if (t ≠ 1) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_0 = 0
end
    
```

fil 1

```

postit_1 = 1
turn = 0
/* sense barrera */
loop: q = postit_0
if (!q) goto cs
t = turn
if (t ≠ 0) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_1 = 0
end
    
```

memòria compartida

```

milk = 0
postit_0 = 0
postit_1 = 0
turn = 0
    
```

fil 0 buffer:p0=1, turn=1

fil 1 buffer:p1=1, turn=0

TOTS DOS A LA SECCIÓ CRÍTICA!

exploració exhaustiva

acaba amb 1 litre(s): 14 estat(s) final(s)

acaba amb 2 litre(s): 4 estat(s) final(s)

contraexemple més curt: 12 passos (pas 12)

Instruccions atòmiques: test-and-set i compare-and-swap

c

```
int test_and_set(int *p) {    /* atomic */
    int old = *p;  *p = 1;  return old;
}
/* spinlock: busy-waits until the old value is 0 */
void lock(int *l)    { while (test_and_set(l)) ; }
void unlock(int *l) { *l = 0; }
```

- x86: `xchg`, `lock cmpxchg`, `lock xadd`; ARM64: `ldxr` / `stxr` o `cas`
- L'espera activa malgasta CPU i pot provocar una **inversió de prioritat**
- Un pany sense espera? No. Sense espera activa? En part →

Espera activa limitada, avui: el futex

Sortida real

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
  0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00    0.223730    27    8013    3116 futex
```

- Mutex lliure → una instrucció atòmica a l'espai d'usuari, **sense crida al sistema**
- Mutex ocupat → `futex(WAIT)` : el nucli adorm el fil i el desperta amb l'`unlock`
- `pthread_spin_lock` no dorm mai (espera activa): només per a seccions molt curtes en què dormir costaria més que girar

Panys: pthread_mutex_lock / unlock

C tds/td6/tickets_mutex.c

```
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

...
for (;;) {
    pthread_mutex_lock(&lock);
    if (tickets ≤ 0) {                               /* checked w.
        pthread_mutex_unlock(&lock);
        break;
    }
    sold++;
    sched_yield();
    tickets--;
    pthread_mutex_unlock(&lock);
}
```

O dinàmic: pthread_mutex_init(&m, attr)
 (pthread_mutexattr_t; NULL = valors per defecte) ... pthread_mutex_destroy(&m).

Sortida real

```
$ ./tickets_race 100000 | tail -3
...
103195 tickets sold out of 100000 (tickets = -7)
```

Sortida real

```
$ ./tickets_mutex 100000 | tail -3
...
100000 tickets sold out of 100000 (tickets = 0)
```

Correcció (diapositiva 162): retorna 0 o el número d'error (EDEADLK només en mutex amb comprovació d'errors), no -1.

Semàfors (Dijkstra, finals dels anys 1960)

Un enter ≥ 0 amb dues operacions atòmiques: **P** (*proberen*: espera que sigui positiu i resta 1) i **V** (*verhogen*: suma 1 i desperta).

- Valor inicial 1 → pany
- Valor inicial 0 → esperar un altre fil
- Valor inicial k → com a màxim k alhora

c

```
int sem_init(sem_t *s, int pshared, unsigned value);
int sem_destroy(sem_t *s); /* unnamed */
sem_t *sem_open(const char *name, int flags /* O_CREAT [| O_EXCL] */,
                mode_t mode, unsigned value); /* SEM_FAILED */
int sem_close(sem_t *s); /* named: this process */
int sem_unlink(const char *name); /* named: the name */
int sem_wait(sem_t *s); /* P */
int sem_post(sem_t *s); /* V */
```

Correcció (diapositiva 167): `sem_open` retorna `SEM_FAILED` (no -1) i no té valor per defecte: amb `O_CREAT` cal donar `value`. I `sem_close` no esborra el semàfor: el nom queda a `/dev/shm` fins a `sem_unlink`.

El problema del productor i els consumidors

- Un productor escriu dades en una memòria intermèdia compartida de mida limitada
- Els consumidors les treuen
- El productor espera si la memòria intermèdia és plena; el consumidor, si és buida
- La memòria intermèdia es modifica: cal una secció crítica

Exemples: un pipeline d'ordres (unitat 5),
`gcc ( cpp | cc1 | as | ld )`, filtres
d'imatges, cues de missatges entre serveis.

Sortida real

```
$ ./prodcons_multi 4 100000  
consumer 2: 23918 items  
consumer 3: 26423 items  
consumer 1: 23037 items  
consumer 0: 26622 items  
sum = 5000050000, expected 5000050000: OK
```

Dos semàfors i un mutex, pas a pas

productor

```
in = (in+1) % BUF
v = 32 · en marxa
```

memòria intermèdia (BUF = 8)



out in

empty = 5 full = 0 out_lock: C2

consumidor 1 · 8

```
wait(full)
espera: full = 0
```

consumidor 2 · 5

```
unlock(out_lock)
en marxa
```

consumidor 3 · 9

```
wait(full)
espera: full = 0
```

consumidor 4 · 7

```
lock(out_lock)
espera el mutex
```

els 40 valors: verd = una vegada · vermell = perdut · taronja = repetit



real, prodcons_multi 4 100000: valors per consumidor



Variables de condició

C

```
pthread_mutex_lock(&mutex);
while (!condition)           /* while, not if */
    pthread_cond_wait(&cond, &mutex);
/* condition is true, mutex held */
pthread_mutex_unlock(&mutex);
/* whoever changes the condition: */
pthread_cond_signal(&cond);   /* wakes one */
pthread_cond_broadcast(&cond); /* wakes all */
```

- Esperar qualsevol condició (no només un comptador)
- `cond_wait` allibera el mutex i dorm de manera **atòmica**, i el torna a agafar en despertar
- `while` i no `if`: hi pot haver despertars espuris o un altre fil pot haver canviat la condició

Interbloquejos

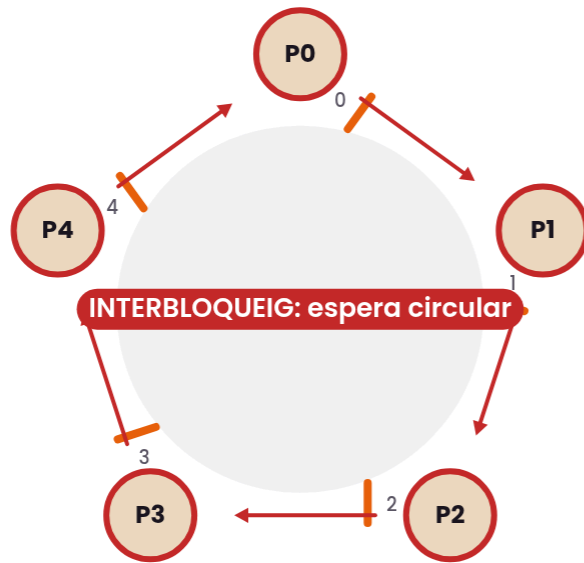
C

```
/* thread 1 */      /* thread 2 */
x.P();              y.P();
y.P();              x.P();  /* ← may never return */
/* ... */           /* ... */
y.V(); x.V();        x.V(); y.V();
```

No passa sempre: depèn de l'execució.
Necessita més d'un recurs i no es resol sense una intervenció externa.

- Recursos: CPU, memòria, disc, dispositius; compartibles o no; expropiables o no
- Exemple: una cruïlla amb preferència, cada cotxe ocupa un tram i espera el següent
- Sortides: fer marxa enrere, treure un recurs (expropiació)... amb risc d' inanició

El sopar dels filòsofs



- pensa
- té gana
- menja
- acabat
- espera (graf)

Condicions de Coffman

- ✓ exclusió mútua
- ✓ retenció i espera
- ✓ no expropiació
- ✓ espera circular

Les quatre condicions i com prevenir-les

1. Exclusió mútua
 2. Retenir recursos mentre s'esperen altres
 3. No expropiació
 4. Espera circular: T_1 espera T_2 , ..., T_n espera T_1
- **Ordre global** d'adquisició (trenca la 4): la correcció del TD6
 - Expropiar (trenca la 3), amb risc d'estat inconsistent
 - Detectar cicles al graf d'assignació; evitar-los amb l'algorisme del banquer
 - Temps límit (`trylock`, `timedlock`): sense garantia

Conclusió

- Problemes: resultats no deterministes, interbloquejos, inanició
- Tota escriptura en un recurs compartit → en una secció crítica
- L'exclusió mútua per programari funciona sobre el paper; en maquinari real, **mutex, semàfors i operacions atòmiques**
- Els interbloquejos són difícils de reproduir i requereixen les quatre condicions

TD6 resol't + repte IA

- threads, tickets_race/mutex/sem
- prodcons, prodcons_multi, philosophers (i la seva versió que s'interbloqueja)
- Nou: create_cost, peterson, lock_cost

Bash

```
make -C tds && tds/tests/run.sh td6
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. Per què `tickets_race` ven més entrades de les que hi ha?
2. Per què falla Peterson en un x86 i què ho corregeix?
3. Quina condició trenca l'ordre global dels bastonets?